Artificial Neural Network (ANN) Report

Heart Disease Prediction

Author	Ishfaque Ahmed
University	Sukkur IBA University
Instructor	Atiqa Aiman
Course	Machine Learning
Dataset	Heart Disease Dataset — Kaggle
Framework	TensorFlow / Keras (Python)
Submitted	April 2026

Table of Contents

Contents

1. Introduction

Heart disease is one of the leading causes of death worldwide. Being able to predict whether a patient has heart disease — based on their health data — can help doctors take action early and save lives. In this assignment, we use an **Artificial Neural Network (ANN)** to make that prediction automatically from patient records.

An ANN is a computer model inspired by the human brain. It learns patterns from data by adjusting its internal connections (called weights) over many training rounds. Once trained, it can look at a new patient's data and predict: **Does this person likely have heart disease or not?**

The dataset used contains various health indicators and risk factors related to heart disease, including demographic, lifestyle, and clinical measurements. The goal is to build a model that can classify each patient as either having or not having heart disease.

The report is structured as follows:

- **Section 2** describes the dataset and its features.
- **Section 3** explains the methodology — preprocessing, architecture, and training.
- **Section 4** presents the results and evaluation metrics.
- **Section 5** concludes with findings and suggestions for improvement.

2. Dataset Description

2. Dataset Description

The dataset used is the **Heart Disease Dataset** from Kaggle:
https://www.kaggle.com/datasets/oktayrdeki/heart-disease

It contains medical records of **10,000 patients** with **21 columns** (20 input features + 1 target label).

2.1 Key Details

Property	Value
Total Samples	10,000 patients
Total Features	20 input features + 1 target column
Target Column	Heart Disease Status (0 = No Disease, 1 = Disease)
Task Type	Binary Classification
Missing Values	Present (handled with mean/mode filling)
Class Split	80% No Disease (8,000) / 20% Disease (2,000)

Table 1: Dataset Summary

2.2 Input Features

- **Age** — The individual's age.
- **Gender** — Male or Female.
- **Blood Pressure** — Systolic blood pressure reading.
- **Cholesterol Level** — Total cholesterol level.
- **Exercise Habits** — Level of physical activity (Low / Medium / High).
- **Smoking** — Whether the individual smokes (Yes / No).
- **Family Heart Disease** — Family history of heart disease (Yes / No).
- **Diabetes** — Whether the individual has diabetes (Yes / No).
- **BMI** — Body Mass Index.
- **High Blood Pressure** — Presence of high blood pressure (Yes / No).
- **Low HDL Cholesterol** — Low HDL indicator (Yes / No).
- **High LDL Cholesterol** — High LDL indicator (Yes / No).

- **Alcohol Consumption** — Intake level (None / Low / Medium / High).
- **Stress Level** — Level of stress (Low / Medium / High).
- **Sleep Hours** — Number of hours the individual sleeps.
- **Sugar Consumption** — Sugar intake level (Low / Medium / High).
- **Triglyceride Level** — Triglyceride level in the blood.
- **Fasting Blood Sugar** — Fasting blood sugar level.
- **CRP Level** — C-reactive protein level (inflammation indicator).
- **Homocysteine Level** — Amino acid level affecting blood vessel health.

Target: *Heart Disease Status* (Yes / No) — the column the model learns to predict.

2.3 Class Distribution

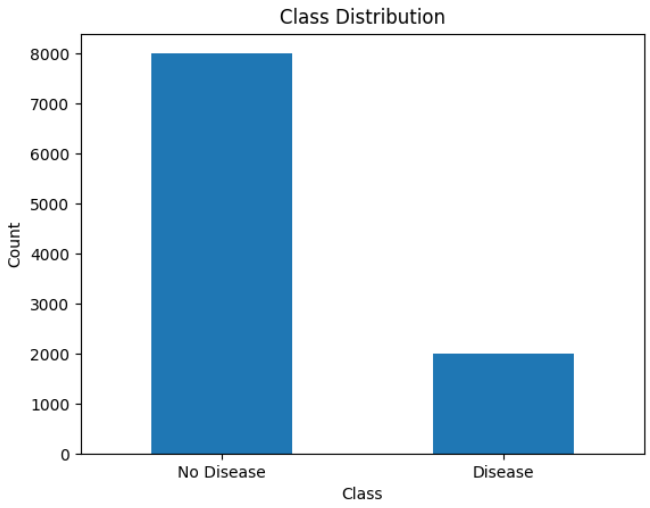

Figure 1: Class Distribution — 80% No Disease vs. 20% Disease (imbalanced dataset)

△ **Important:** This imbalance (80% vs. 20%) is the root cause of the model's poor performance on disease detection. The model learns to simply predict "No Disease" every time and still scores 80% accuracy — but misses ALL actual disease cases. This is explained in detail in Section 4.

2.4 Correlation Matrix

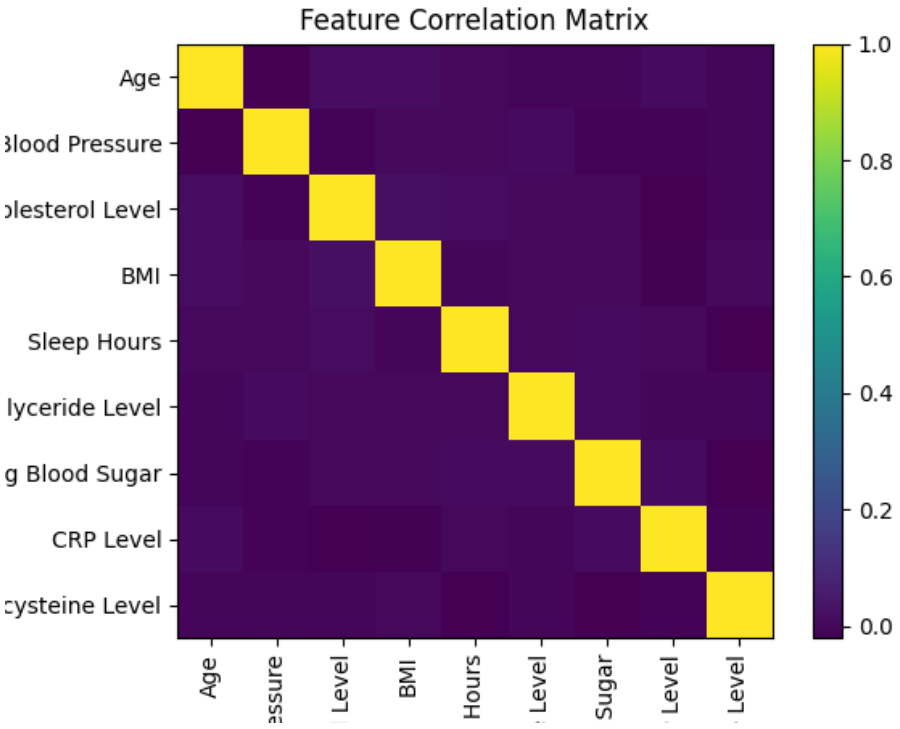

Figure 2: Correlation matrix of all 20 input features

3. Methodology

3. Methodology

3.1 Data Preprocessing

- **Missing values:** Numerical columns filled with their mean value; categorical columns filled with the most common value (mode).
- **Categorical encoding:** Text columns (e.g. 'Male/Female', 'Yes/No') were converted to numbers using Label Encoding so the model can process them.
- **Feature scaling:** All numerical values were standardised (mean = 0, std = 1) using `StandardScaler` so no single feature dominates.
- **Train/Test split:** 80% of data (8,000 rows) for training; 20% (2,000 rows) for testing. A stratified split ensures both sets preserve the same class ratio.

3.2 Exploratory Data Analysis (EDA)

Before building the model, we analysed the data to understand its structure. Key steps included:

- Checking the shape and data types of all columns.
- Visualising feature distributions using histograms.
- Computing a correlation matrix to identify related features.
- Identifying and visualising the class imbalance in the target variable.

3.3 ANN Architecture

The ANN was built using **TensorFlow** and **Keras** with the following structure:

Layer	Neurons	Activation	Note
Input Layer	20	—	One node per feature
Hidden Layer 1	64	ReLU	Dropout (30%)
Hidden Layer 2	32	ReLU	Dropout (20%)
Output Layer	1	Sigmoid	Outputs probability (0 to 1)

Table 2: ANN Architecture

What is Dropout? Dropout randomly switches off some neurons during training. This prevents the model from memorising the training data (overfitting) and forces it to learn more general patterns.

What is ReLU? ReLU (Rectified Linear Unit) outputs the input if it is positive, otherwise zero. It helps the network learn complex, non-linear relationships.

What is Sigmoid? Sigmoid squeezes the output to a value between 0 and 1. Values above 0.5 are treated as 'Disease'; below 0.5 as 'No Disease'.

3.4 Model Training Settings

Setting	Value
Optimizer	Adam (Adaptive Moment Estimation)
Loss Function	Binary Cross-Entropy
Epochs	50
Batch Size	32
Validation Split	10% of training data

Table 3: Training Configuration

4. Results

4. Results

4.1 Training Curves

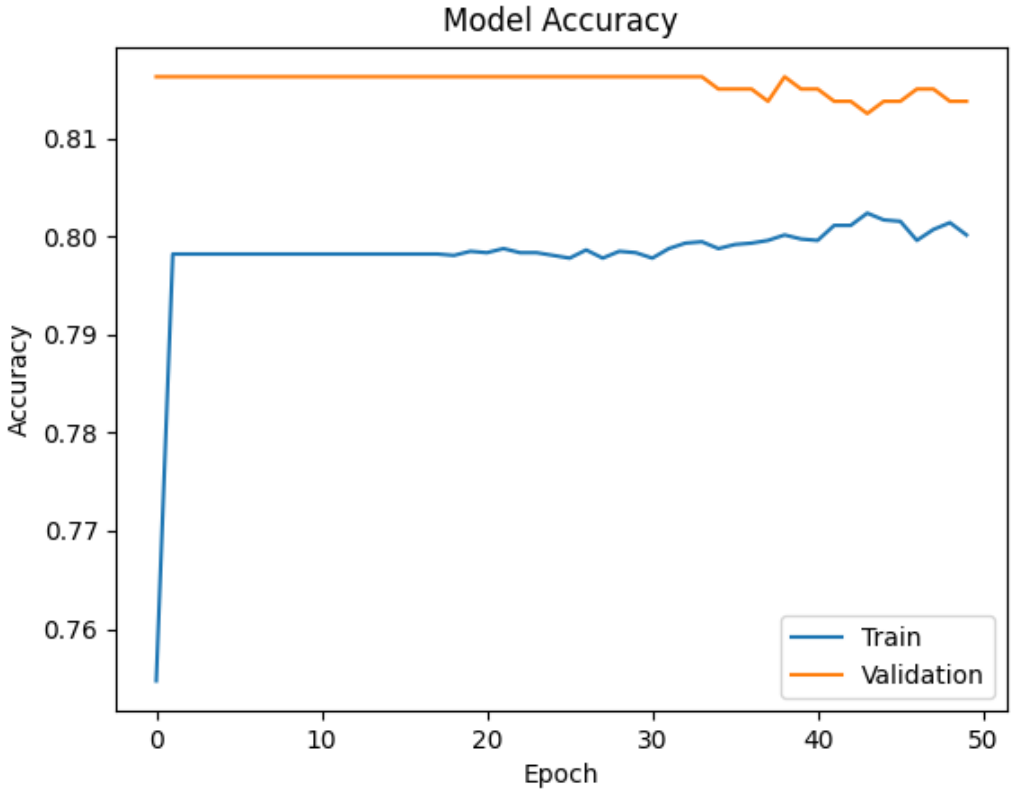

Figure 3: Training and validation **accuracy** over 50 epochs

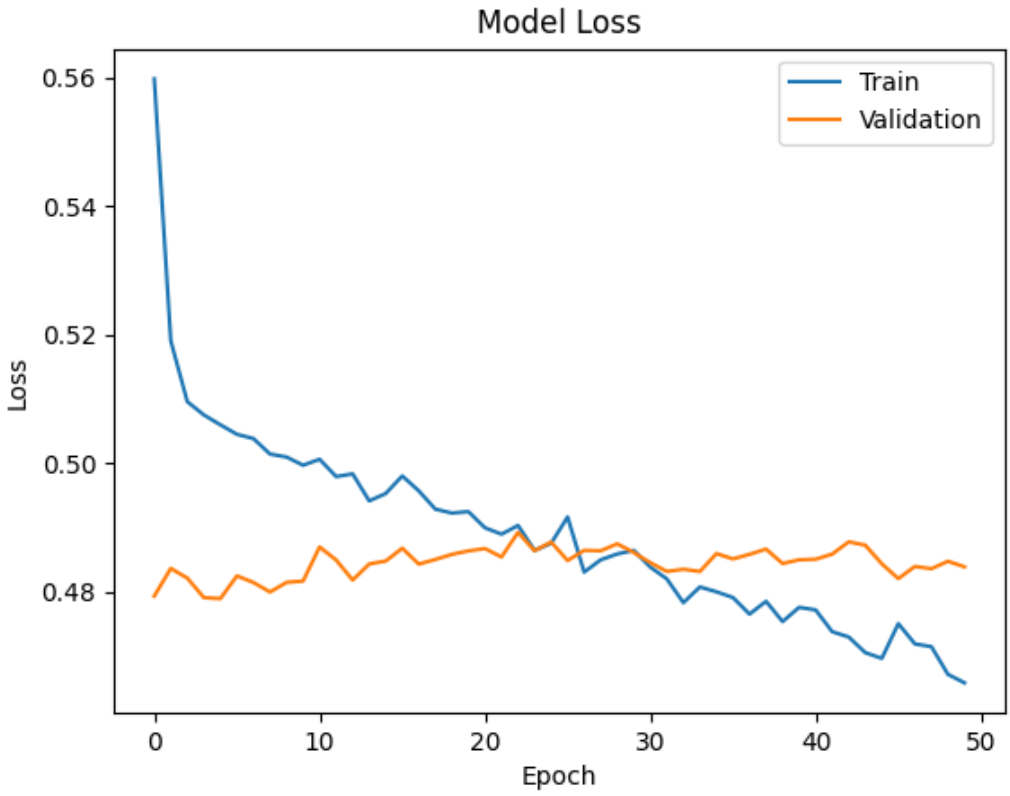

Figure 4: Training and validation **loss** over 50 epochs

The training accuracy quickly rose to about 80% and then stayed flat for the remaining epochs. The validation accuracy also plateaued at around 81–82%. This indicates the model converged early and stopped improving — a sign that it is not learning to distinguish the two classes meaningfully.

4.2 Confusion Matrix

Metric		Value	What it means
True (TP)	**Positives**	0	Disease cases correctly predicted as Disease
True (TN)	**Negatives**	1,600	No Disease correctly predicted as No Disease
False (FP)	**Positives**	0	No Disease wrongly predicted as Disease
False (FN)	**Negatives**	400	Disease cases MISSED — predicted as No Disease

Table 4: Confusion Matrix Breakdown

4.3 Classification Report

Class	Precision	Recall	F1-Score	Support
No Disease	0.80	1.00	0.89	1,600
Disease	0.00	0.00	0.00	400
Overall Accuracy		**0.80**		2,000

Table 5: Classification Report

4.4 Discussion of Results

What the 80% Accuracy Really Means

At first glance, 80% accuracy sounds good. But looking at the confusion matrix and classification report reveals a serious problem: **the model predicted ZERO disease cases correctly.** It simply learned to say 'No Disease' for every single patient.

△ **Warning:** The model missed all 400 actual heart disease patients in the test set. In a medical context, this is extremely dangerous — these are people who needed attention but were incorrectly told they are healthy.

Why did this happen? Because 80% of the data has 'No Disease'. The model discovered a shortcut: always predicting 'No Disease' gives 80% accuracy without learning anything useful. This is the **class imbalance problem**.

Overfitting vs. Underfitting

The training and validation accuracy were almost identical (≈80%), and the loss curves stayed close together. This means the model is **not overfitting**. However, it is also not learning properly — it is **underfitting** the minority class (Disease), since it never predicts it.

Suggested Improvements

Class Weights: Tell the model that Disease cases are more important. In Keras: `class_weight = {0: 1.0, 1: 4.0}` — this penalises the model more when it misses a disease case.

SMOTE (Oversampling): Synthetically create more Disease samples so the training set is balanced (50/50). This is done before training using the `imbalanced-learn` library.

Lower Decision Threshold: Instead of 0.5, use a lower cutoff such as 0.3. This flags more patients as potential disease cases, reducing missed diagnoses.

More Layers / Neurons: Add a third hidden layer or increase neurons in existing layers to give the model more capacity to find subtle patterns.

More Epochs with Early Stopping: Training for 100–200 epochs with early stopping (stop when validation loss stops improving) may allow the model to learn more.

5. Conclusion

5. Conclusion

In this assignment, we built an Artificial Neural Network to predict heart disease using a 10,000-patient Kaggle dataset. The model was trained with TensorFlow/Keras using a two-hidden-layer architecture with Dropout regularisation, the Adam optimiser, and binary cross-entropy loss.

The model achieved an overall accuracy of **80%**, but this number is misleading. The classification report revealed that the model predicted zero disease cases correctly, failing entirely on the class that matters most in a medical context.

The root cause is **class imbalance**: the dataset has 4× more 'No Disease' samples than 'Disease' samples. The model learned to always predict the majority class — a very common real-world problem in medical data.

To fix this, future work should apply class weighting, SMOTE oversampling, or threshold tuning. These changes would allow the model to actually detect disease, making it genuinely useful rather than just statistically accurate.

Summary of Results

- Overall Accuracy: **80%** (misleading due to class imbalance)
- Disease Recall: **0%** — all 400 disease cases were missed
- No Disease Recall: **100%** — all non-disease cases predicted correctly
- Root Cause: **Class imbalance (80/20 split)**
- Key Fix: Apply class weights or SMOTE before retraining